COMP306 – Database Systems

The Starlight Carnival Management System Project Documentation

A comprehensive Documentation of our COMP306 Project drafted with the assistance of AI due to time constraints

by Group G

University of KwaZulu-Natal



Project Formalities

Amusement Park Management System

The Starlight Carnival

Group G

Group Members

Member Name	Student ID	Responsibility
Jawad Kachbal	223037045	Relational Schema
Joshua Blom	224039813	EERD & Business Scenario
Mahomed Valli Mahomed	224005878	SQL Design & Implementation / Front End
Abiel Govender	224058851	SQL Design & Implementation
Andiswa Hlengwa	221002430	Business Scenario & Front End

Table of Contents

Project Formalities 1

How to Run the Project 2

Prerequisites 3

Step 1 — Import the Database 3

Step 2 — Open the Project in IntelliJ 3

Step 3 — Attach the MySQL JDBC Driver 3

..... 4

Step 4 — Run the Application 4

Step 5 — Log In 5

Front End — Java Swing GUI 5

Architecture Overview 5

Role-Based Routing 6

Generic CRUD via EntityConfig 6

CRUD Operations 6

Reports Screen — Three+ Complex Queries 7

Role-Specific Frames 7

Database Connectivity and Transactions 8
UI Theme and Usability 8
AI Usage Declaration 9

Table of Figures

FIGURE 1 IMAGE 1 SHOWCASING THE PROCESS IN INTELLIJ 3
FIGURE 3 IMAGE 2 SHOWCASING THE PROCESS IN INTELLIJ 4
FIGURE 2 IMAGE 3 SHOWCASING THE PROCESS IN INTELLIJ 4

NOTE – PLEASE SEE README FOR A SUMMARY AND FOLDER STRUCTURE

How to Run the Project

This section explains how to set up the MySQL database, open the project in IntelliJ IDEA, attach the MySQL JDBC driver, and launch the Java Swing application.

Prerequisites

- MySQL Server 8.0 or higher with MySQL Shell installed.
- Java Development Kit 17 or higher (the screenshots use JDK 25).
- IntelliJ IDEA (Community or Ultimate).
- The submitted project folder containing /src, /lib/mysql-connector-j.jar, starlight_carnival.sql, and the .bat runner.

Step 1 — Import the Database

The SQL script must be loaded into MySQL before launching the application. If it has not been loaded, the login screen will report that required tables were not found.

1. Open MySQL Shell.
2. Connect to your local MySQL server:

```
\connect root@localhost
```

Enter your local MySQL root password when prompted.

3. Create and select the database:

```
CREATE DATABASE starlight_carnival;

\use starlight_carnival
```

4. Load the SQL script — replace the path with the location of starlight_carnival.sql on your machine:

```
SOURCE C:/Users/YourName/Desktop/Starlight Carnival/starlight_carnival.sql;
```

The script creates all 22 tables, three views, two stored procedures, all triggers, four database user accounts, and seeds the database with sample data.

Step 2 — Open the Project in IntelliJ

1. Launch IntelliJ IDEA and choose Open.
2. Navigate to the extracted Starlight Carnival folder and select it.
3. Wait for IntelliJ to finish indexing the project. The folder structure will show src/starlightcarnival containing the Java source files and lib/ containing the MySQL JDBC connector.

Step 3 — Attach the MySQL JDBC Driver

IntelliJ does not automatically place lib/mysql-connector-j.jar onto the classpath when the project is opened for the first time. This step adds the driver as a project dependency so the application can connect to MySQL.

1. In the project tree on the left, right-click the top-level project node (Starlight

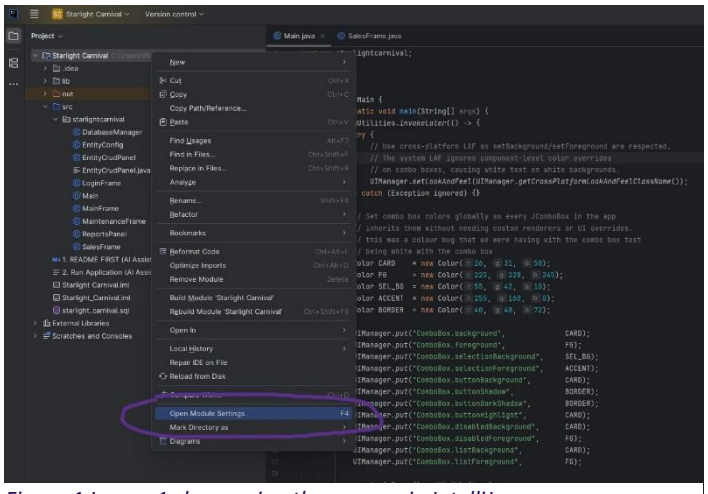


Figure 1 Image 1 showcasing the process in IntelliJ

Carnival) and choose Open Module Settings (shortcut: F4). This is shown in image 1 of the setup screenshots.

2. In the Project Structure window that opens, select Modules in the left sidebar, then click the Dependencies tab on the right.
3. Click the + (Add) button above the dependencies list and choose 1. JARs or Directories… from the dropdown menu.
4. In the file browser, navigate to the lib folder inside the Starlight Carnival project directory and select mysql-connector-j.jar. Click Open.
5. The driver will now appear in the dependencies list with Scope set to Compile. Confirm that both Project SDK 25 (Oracle OpenJDK) and mysql-connector-j.jar are listed, then click Apply followed by OK.
6.

After applying the changes, IntelliJ may re-index briefly. Once that finishes, the project is ready to run

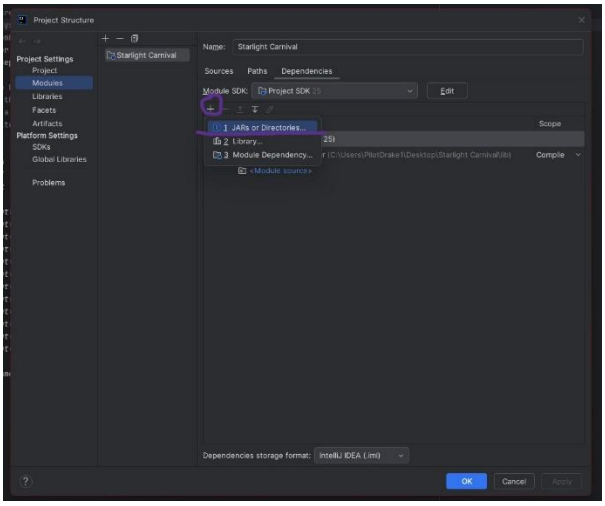


Figure 2 Image 2 showcasing the process in IntelliJ

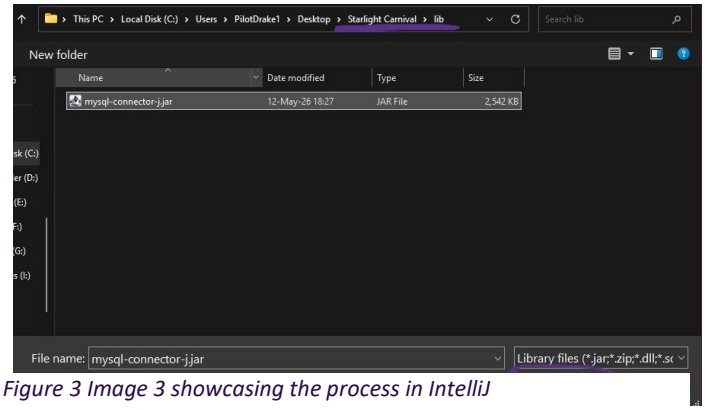


Figure 3 Image 3 showcasing the process in IntelliJ

Step 4 — Run the Application

There are two ways to launch the application:

Option A — From IntelliJ

1. Open src/starlightcarnival/Main.java.
2. Click the green Run arrow next to the main method, or press Shift + F10.

Option B — From the .bat file

If the database has been loaded, the .bat file can launch the application without opening IntelliJ at all.

1. Double-click 2. Run Application (AI Assisted).bat in the project root.

Note — the .bat file compiles the Java sources against the bundled MySQL connector and starts the application directly. If you receive a login error, confirm the SQL script has been loaded into MySQL.

Step 5 — Log In

The SQL script provisions four MySQL accounts that each open a different role-specific GUI:

Username	Password	Role	Available Screens
starlight_admin	admin123	Admin	Full CRUD on every entity plus Reports
starlight_sales	sales123	Sales	Customer admission and access-pass workflow
starlight_maintenance	maint123	Maintenance	Logging and reviewing maintenance jobs
starlight_guest	guest123	Guest	Read-only view of every screen and report

Front End — Java Swing GUI

Architecture Overview

The front end is a desktop application built with Java Swing on the standard JDK runtime. It talks to MySQL through the official MySQL Connector/J JDBC driver. The codebase is organised into eight classes, each with a single, well-defined responsibility:

Class	Responsibility
Main.java	Application entry point. Configures the Swing look-and-feel and global JComboBox colours, then opens the login screen.
LoginFrame.java	Username/password screen. Calls DatabaseManager.connect(...) and routes the user to the appropriate frame based on the matched role.
DatabaseManager.java	Singleton wrapper around the JDBC connection. Provides query(), update() and runInTransaction() helpers, infers the role from the username, and validates that the schema has been loaded.
EntityConfig.java	Declarative metadata for every entity — table name, primary key, field list, dropdown options, foreign keys, subtype hierarchies, insert permissions. Drives both the form and the data grid generically.
EntityCrudPanel.java	Generic Create / Read / Update / Delete panel built from an EntityConfig. Renders the correct input type per field (text, date, dropdown, foreign-key combo, boolean checkbox), supports search/filter, and handles supertype + subtype inserts as a single logical operation.
MainFrame.java	Admin / guest shell. Sidebar of entities and a Reports tab; in guest mode the panel is rendered read-only.
SalesFrame.java	Four-step pipeline for the sales role — pick customer, log admission, create access pass, summary.
MaintenanceFrame.java	Two-screen workflow for the maintenance role — log a new job and browse existing logs.
ReportsPanel.java	Runs the five complex SQL reports and renders each as a card with a table and summary statistics.

Role-Based Routing

LoginFrame asks DatabaseManager.connect() to authenticate against MySQL. The username determines the role — accounts containing 'admin' or 'root' resolve to ADMIN, 'sales' to SALES, 'maintenance' or 'maint' to MAINTENANCE, and anything else (including the guest account) to GUEST. The role then chooses which window to open:

- **ADMIN** → MainFrame in read-write mode (full CRUD + Reports).
- **SALES** → SalesFrame (the four-step admission and access-pass workflow).
- **MAINTENANCE** → MaintenanceFrame (log a new job, browse logs).
- **GUEST** → MainFrame in read-only mode (every CRUD panel is disabled, but Reports remain available).

Because the MySQL account itself has only the privileges its role needs, a malicious or buggy client cannot bypass these rules — the database will refuse a forbidden statement regardless of what the GUI does.

Generic CRUD via EntityConfig

Rather than writing a bespoke form for each of the 21 entities, the system declares each entity once in EntityConfig.ALL and lets EntityCrudPanel build the form, the data grid and the SQL at runtime.

EntityConfig captures, per entity:

- Table name, primary key(s) and a human-readable label.
- Each editable field's key, label, type (TEXT, NUMBER, DATE, TIME, DATETIME, SELECT, BOOLEAN, TEXTAREA) and required flag.
- Dropdown options for fixed value sets (e.g. card_status of Active/Inactive/Blocked).
- Foreign-key bindings — a foreign-key field becomes a combo box populated by querying the referenced table for a value (ID) and label (display text).
- Subtype configuration — if the entity is a supertype (Employee, Attraction, AccessPass), each subtype is declared with its own discriminator value, table name and field list.
- allowInsert flag — set to false where row creation must go through a richer workflow (e.g. ATTRACTION rows are inserted as part of the subtype-aware Ride / Show / Stall flow rather than directly).

EntityCrudPanel reads this metadata and renders the form, validates inputs, runs CRUD operations and refreshes the grid. Adding a new entity to the application is therefore a matter of writing one EntityConfig entry, not building a new screen.

CRUD Operations

Create, Read, Update and Delete are implemented as follows:

Create

The form is rendered from the EntityConfig field list. Foreign keys appear as combo boxes whose values come from a real-time query against the referenced table. For supertypes, an additional combo box selects the subtype, and the form expands to include the subtype's own fields. Insertion of a supertype + subtype row is wrapped in a transaction (DatabaseManager.runInTransaction(...)) so the parent row and the subtype row commit together or not at all.

Read

The data grid runs a SELECT against the table (or a view, where appropriate) and renders the result through a custom row renderer that alternates row colours and respects the application's dark theme.

Update

Selecting a row in the grid loads its values back into the form. Saving runs an UPDATE keyed on the primary key; for supertypes, both the parent UPDATE and the subtype UPDATE are issued inside a transaction.

Delete

A confirmation dialog is shown before issuing DELETE. Because the SQL schema is configured with ON DELETE CASCADE on subtypes and dependent rows, deleting an Employee (for example) automatically removes the matching Operator/Technician/Vendor row and any other dependents.

Reports Screen — Three+ Complex Queries

ReportsPanel runs five reports concurrently using SwingWorker so the UI never blocks while the queries execute. Each report is a join across three or more tables (the project brief requires at least three), is rendered as a card, and shows the SQL statement that produced it. The reports satisfy and exceed the assignment requirement of three GUI screens of significant complexity.

Report 1 — Revenue by Location and Admission Type

Joins ADMISSION, LOCATION and CUSTOMER. Groups by venue_name, admission_type and age_group; returns admission count, total revenue and unique customer count, sorted by revenue. Answers: how does revenue break down across the tour?

Report 2 — Ride Maintenance Summary

Joins RIDE, ATTRACTION and MAINTENANCE_LOG. Returns one row per ride with the count of maintenance jobs, total parts cost and number of technicians who worked on it, sorted by parts cost. Answers: which rides are eating maintenance budget?

Report 3 — Products by Stall Stock

Joins PRODUCT, STALL_STOCK, STALL and ATTRACTION. Returns the stall a product is sold at and the total quantity in stock. Answers: what is on sale where, and how much of it is left?

Report 4 — Employee Distribution by Location

Joins EMPLOYEE, LOCATION and all five Employee subtype tables. Returns employee count, average wage and earliest hire date per (location, employee_type) pair. The subtype tables are used to confirm that every Employee has been correctly placed into exactly one subtype — rows reading 'Unassigned' would indicate a data integrity problem. Answers: what does the staff look like at each stop on the tour?

Report 5 — Customer Loyalty and Spending

Joins CUSTOMER, TRANSACTION and PAYMENT. Returns the top 50 customers by total amount spent, with their card balance, loyalty points, transaction count and total paid. Answers: who are the most valuable customers and is their loyalty points balance consistent with their spend?

Role-Specific Frames

SalesFrame

A four-step pipeline that walks a sales operator through onboarding a guest:

1. Identify the customer — search by ID, name or phone, or register a brand-new customer on the spot (which also generates a smart card in active status).
2. Log the admission — pick a location, the system computes the admission type from the customer's age group, the operator enters the entry fee.
3. Create an access pass (optional) — choose Ride Fast Pass or Show Reservation. The flow shows only valid attractions of the chosen type and writes both the ACCESS_PASS row and its subtype row inside a single transaction.
4. Summary screen — confirms what was created and offers a Back to Home button.

Every step writes to the database through prepared statements, and the entire admission-plus-pass operation is transactional so a half-completed sale is never left behind.

MaintenanceFrame

Two screens for the maintenance role:

- New Maintenance Record — pick a ride, pick the technician on shift, describe the issue, describe the action taken, enter the parts cost. The ride list is filtered to RIDE subtype rows only, and the technician list is filtered to TECHNICIAN subtype rows only.

View Maintenance Records — a sortable grid showing every log entry with ride name and technician name resolved through joins, drawn from the VW_MAINTENANCE_OVERVIEW view

Database Connectivity and Transactions

DatabaseManager is a singleton holding one JDBC Connection per running JVM. It exposes three high-level helpers used everywhere in the application:

- `query(sql, params...)` — runs a SELECT through a PreparedStatement and returns a List<Map<String, Object>>, isolating call sites from JDBC's ResultSet API.
- `update(sql, params...)` — runs INSERT/UPDATE/DELETE through a PreparedStatement and returns the generated key when there is one.
- `runInTransaction(work)` — runs a block of statements under setAutoCommit(false) and commits or rolls back atomically. Used for supertype+subtype inserts and for the admission+pass sale in SalesFrame.

All queries are parameterised — concatenated SQL is never used — which removes the risk of SQL injection from user-supplied values.

UI Theme and Usability

Every frame shares the same dark theme defined as constants on Main and each Frame class (CARD, FG, SEL_BG, ACCENT, BORDER). Global JComboBox colours are pushed into UIManager at startup so dropdowns render consistently across the platform's look-and-feel — this fixes a white-on-white text bug that appeared on Windows under the system L&F. Tables use alternating row colours, primary buttons carry an accent colour, and status messages on the login and pipeline screens are colour-coded green for success and red for errors.

AI Usage Declaration

Group G used AI tools during the project for the purposes listed below. All design decisions, business logic and the architecture of the application were made by the group; AI assistance was confined to review, polish and routine code support.

- Plausibility checking — confirming that the business scenario hangs together and that no relationships were missing.
- Assistance with polishing entity attribute details for clarity and consistency.
- Grammar checking on the written documents.
- Code review and routine assistance with the Java Swing front-end, especially around generic CRUD form generation and styling consistency across frames.
- Documentation drafting — this report was produced with AI assistance from the project source files, source code, the EERD, the relational schema document, and the Phase 1 business scenario already authored by the group.